

Name:
Enrolment No:



MANIPAL UNIVERSITY
JAIPUR

Odd Semester Makeup Examination, December 2025
Faculty of Engineering, School of Computer Science and Engineering
Department of Computer Science and Engineering
B.Tech.-CSE

Course Code: CSE2103

Course: Computer Organization and Architecture

Semester: III

Time: 3 hrs.

Max. Marks: 80

Solution

SECTION A

S.No.		Marks	CO
Q A1	What are the various factors that affect the performance of a computer system? The performance of a computer system is affected by factors such as CPU clock speed, number of cores, instruction set and CPI, cache memory size and hierarchy, main memory speed, I/O performance, and compiler efficiency.	[2]	CO1
Q A2	What is the syntax and use of the relative addressing mode? Syntax: Effective Address (EA) = PC + Offset Use: In relative addressing mode, the operand address is obtained by adding a signed offset to the current Program Counter (PC). It is commonly used for branch and jump instructions to support position-independent code.	[2]	CO2
Q A3	Write an assembly language program for a single-accumulator organization to evaluate $Z = (A+B) * (C + D)$. LOAD A ADD B STORE T LOAD C ADD D MUL T STORE Z	[2]	CO2
Q A4	What is the key advantage of associative mapping over direct mapping? The key advantage of associative mapping is that any block of main memory can be placed in any cache line , which significantly reduces cache conflicts and miss rate compared to direct mapping.	[2]	CO3
Q A5	Write the expression for gate delay in n-bit array multiplier. $6(n-1)-1$	[2]	CO4
SECTION B			
Q B1	A program consists of four instruction types A, B, C, and D. Instruction A has a CPI of 1.3 and accounts for 60% of the instructions. Instruction B has a CPI of 2.2 and occurs 10% of the time.	[6]	CO1

	Instruction C has a CPI of 2.0 with a frequency of 20%, and instruction D has a CPI of 2.8 with a frequency of 10%. The CPU designer proposes two options: (a) reduce the CPI of instruction A to 1.1, or (b) reduce the CPI of instructions B and D to 1.6. Which option gives better performance? A program has four instruction types A, B, C and D with the following CPI and instruction mix: Instruction A: CPI = 1.3, Frequency = 60% Instruction B: CPI = 2.2, Frequency = 10% Instruction C: CPI = 2.0, Frequency = 20% Instruction D: CPI = 2.8, Frequency = 10% Average CPI before improvement: Average CPI $= (1.3 \times 0.6) + (2.2 \times 0.1) + (2.0 \times 0.2) + (2.8 \times 0.1)$ $= 0.78 + 0.22 + 0.40 + 0.28$ $= \mathbf{1.68}$ Option (a): Reduce CPI of instruction A to 1.1 New Average CPI $= (1.1 \times 0.6) + (2.2 \times 0.1) + (2.0 \times 0.2) + (2.8 \times 0.1)$ $= 0.66 + 0.22 + 0.40 + 0.28$ $= \mathbf{1.56}$ Option (b): Reduce CPI of instructions B and D to 1.6 New Average CPI $= (1.3 \times 0.6) + (1.6 \times 0.1) + (2.0 \times 0.2) + (1.6 \times 0.1)$ $= 0.78 + 0.16 + 0.40 + 0.16$ $= \mathbf{1.50}$ Conclusion: Since the average CPI is lower in Option (b), reducing the CPI of instructions B and D gives better performance.		
Q B2	Illustrate architecture of a vector processor with its advantages. A vector processor is designed to perform operations on one-dimensional arrays (vectors) of data using a single instruction. Main Components: Vector Register File (VRF) - Contains a set of vector registers. - Each vector register holds multiple data elements. Vector Functional Units - Dedicated arithmetic units for vector operations such as: - Vector add - Vector multiply - Vector divide	[6]	CO1

	▪ Vector compare 3. Vector Control Unit ○ Decodes vector instructions. ○ Generates control signals for vector operations. 4. Scalar Processor ○ Executes scalar instructions. ○ Handles loop control, branching, and address calculations. 5. Memory System ○ Supports high-bandwidth memory access. ○ Allows vector load and store operations. Vector Control Unit Vector Register File Vector ALU Vector MUL Memory Advantages of Vector Processors 1. High performance for scientific and numerical applications 2. Reduced instruction fetch and decode overhead 3. Better utilization of pipelined hardware 4. Efficient handling of loops and array operations 5. Simpler control compared to superscalar processors 6. High throughput due to parallel element processing		
Q B3	Suppose multiple devices capable of generating interrupts are connected to a CPU. Describe two possible methods the CPU can use to identify the interrupting device and obtain the starting address of the corresponding Interrupt Service Routine (ISR). 1. Vectored Interrupts The interrupting device supplies an interrupt vector number. CPU uses this vector to directly obtain ISR address from a table.	[6]	CO3

	• Fast and efficient, commonly used in modern systems.		
	2. Polling (Non-vectorized Interrupts) • CPU checks devices one by one to identify the interrupt source. • ISR starting address is obtained after identifying the device. • Simple but slow method.		
Q B4	Consider a 4-way set associative cache of size 32 KB with a block size of 128 bytes. If the main memory size is 512 KB, determine number of bits in the tag and size of the tag directory. Given: Cache size = 32 KB Block size = 128 bytes Cache type = 4-way set associative Main memory size = 512 KB	[6]	CO3
	Step 1: Number of cache blocks Cache size = 32 KB = $32 \times 1024 = 32768$ bytes Number of blocks $= 32768 / 128$ $= \mathbf{256 \text{ blocks}}$		
	Step 2: Number of sets 4-way set associative cache Number of sets $= 256 / 4$ $= \mathbf{64 \text{ sets}}$ Index bits $= \log_2 64$ $= \mathbf{6 \text{ bits}}$		
	Step 3: Block offset bits Block size = 128 bytes Offset bits $= \log_2 128$ $= \mathbf{7 \text{ bits}}$		
	Step 4: Main memory address bits Main memory size = 512 KB $= 512 \times 1024 \text{ bytes}$ $= 2^{19} \text{ bytes}$ Main memory address bits $= \mathbf{19 \text{ bits}}$		

	Step 5: Tag bits Tag bits = Total address bits – (Index bits + Offset bits) = 19 – (6 + 7) = 6 bits Step 6: Size of tag directory Total cache lines = 256 Tag directory size = 256 × 6 = 1536 bits Final Answer: Number of tag bits = 6 bits Size of tag directory = 1536 bits		
Q B5	Apply the bit-pair recoding algorithm to multiply the 6-bit 2's complement numbers A = 111111 (multiplicand) and B = 101010 (multiplier). The image shows handwritten work on a grid background. At the top, it identifies A = 111111 as (-1) and B = 101010 as (-21). It then shows the bit patterns for Booth recoding: A = 000000, B = 000001, and their Booth recoded forms: A = 111111, B = 000001. The Booth recoded forms are then multiplied to get the result 00000010110, which is identified as (+21). Below this, the Bit-pair recoding algorithm is shown. It identifies A = 111111 as -1 and B = 101010 as -2. The Bit-pair recoded forms are shown as A = 000000000010, B = 0000000001, and their product is 0000000010110, identified as (+21).	[6]	CO4

SECTION-C

Q C1	Consider a byte-addressable machine with a 32-bit word size. An array of n 32-bit integers is stored in consecutive memory locations starting at address ARR, and the value of n is stored as a 32-bit integer at memory location N. Write an assembly language program to find the maximum element of the array and store the result in memory location MAX, without using any additional array or temporary storage other than registers. <pre> LOAD R1, N ; R1 ← n (number of elements) LOAD R2, ARR ; R2 ← first array element (max = ARR[0]) ADD R3, ARR, #4 ; R3 ← address of next element (ARR + 4) LOOP: SUB R1, R1, #1 ; decrement count BEQ DONE ; if all elements checked, exit loop LOAD R0, 0(R3) ; R0 ← current array element CMP R0, R2 ; compare current element with max BLE SKIP ; if R0 ≤ R2, skip update MOVE R2, R0 ; update max value SKIP: ADD R3, R3, #4 ; move to next array element BR LOOP ; repeat loop DONE: STORE R2, MAX ; store maximum value in MAX HALT </pre> Explanation (for answer writing) • Register R2 holds the current maximum value. • Register R3 points to the next array element. • Register R1 stores the remaining number of elements. • The program scans the array once, compares each element with the current maximum, and updates it if required. • No additional memory locations or arrays are used.	[10]	CO2												
C2	A processor uses a hardwired control unit. It supports four control signals S_0, S_1, S_2, S_3 and an instruction set of up to 256 instructions. Each instruction is executed using a sequence of up to 8 control steps generated by a timing unit. Consider three sample instructions I_1, I_2 and I_3. Each of these instructions requires four microoperations at time steps T_1, T_2, T_3 and T_4. The control signals required at each time step for the execution of these instructions are given in the table below: <table border="1" data-bbox="495 1724 1013 1877"> <thead> <tr> <th>Time Step</th><th>I_1</th><th>I_2</th><th>I_3</th></tr> </thead> <tbody> <tr> <td>T_1</td><td>S_0</td><td>S_0</td><td>S_0</td></tr> <tr> <td>T_2</td><td>S_2, S_3</td><td>S_2</td><td>S_1, S_2</td></tr> </tbody> </table>	Time Step	I_1	I_2	I_3	T_1	S_0	S_0	S_0	T_2	S_2, S_3	S_2	S_1, S_2	[10]	CO5
Time Step	I_1	I_2	I_3												
T_1	S_0	S_0	S_0												
T_2	S_2, S_3	S_2	S_1, S_2												

		T ₃	S ₁	S ₃	S ₁ , S ₃		
		T ₄	S ₀	S ₂ , S ₃	S ₀ , S ₂		
	(a) Using the above table, derive the Boolean logic expressions required to generate control signals S ₀ and S ₁ in terms of instruction decoder outputs and timing signals. Also, draw the corresponding hardwired control logic circuit. (b) Assuming the processor supports up to 256 instructions and that control sequences may last up to 8-time steps, determine: 1. The required size of the instruction decoder 2. The required size of the control-step counter 3. The required size of the step decoder Justify your answers clearly. (a) Boolean expressions for S₀ and S₁ From the given table: Control Signal S₀ - I₁ at T₁ and T₄ - I₂ at T₁ - I₃ at T₁ and T₄ So, S0 = I1·T1 + I1·T4 + I2·T1 + I3·T1 + I3·T4 Or factorized form: S0 = T1·(I1 + I2 + I3) + T4·(I1 + I3) Control Signal S₁ - I₃ at T₂ - I₁ at T₃ - I₃ at T₃ So, S1 = I3·T2 + I1·T3 + I3·T3 Or factorized form: S1 = I3·T2 + T3·(I1 + I3) (b) Instruction decoder: 8-to-256 Control-step counter: 3-bit Step decoder: 3-to-8						
	Section-D						
Q D1	Show the block diagram of a 16-bit carry look-ahead (CLA) adder constructed by cascading four 4-bit CLA adders. Each 4-bit CLA uses first-level Generate (G _i) and Propagate (P _i) functions to compute the carries within the block. Answer the following:	[10]	CO4				

	(a) Write the carry expressions for C_4, C_8, C_{12}, and C_{16} for the 16-bit adder using the appropriate group Generate and Propagate terms. (b) Compute the total gate delay required to generate C_{16} and the sum bit S_{15}. Ans (a) Expressions for Carry are as follows. $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$ $C_8 = G_7 + P_7G_6 + P_7P_6G_5 + P_7P_6P_5G_4 + P_7P_6P_5P_4C_4$ $C_{12} = G_{11} + P_{11}G_{10} + P_{11}P_{10}G_9 + P_{11}P_{10}P_9G_8 + P_{11}P_{10}P_9P_8C_8$ $C_{16} = G_{15} + P_{15}G_{14} + P_{15}P_{14}G_{13} + P_{15}P_{14}P_{13}G_{12} + P_{15}P_{14}P_{13}P_{12}C_{12}$ Ans (b) For Carry $C_{16} = 3 + 3 * 2 = 9$ GD For Sum $S_{15} = 10$ GD																																																																																																																																																			
Q D2	Consider a 4-stage pipeline processor with stages S1, S2, S3, and S4. The number of cycles required by instructions I1, I2, I3, and I4 in each stage is given below: <table><tr><th>Instruction</th><th>S1</th><th>S2</th><th>S3</th><th>S4</th></tr><tr><td>I1</td><td>2</td><td>1</td><td>1</td><td>1</td></tr><tr><td>I2</td><td>1</td><td>3</td><td>2</td><td>2</td></tr><tr><td>I3</td><td>2</td><td>1</td><td>1</td><td>3</td></tr><tr><td>I4</td><td>1</td><td>2</td><td>2</td><td>2</td></tr></table> Assume the following loop executes: <pre>for (i = 1 to 2) { I1; I2; I3; I4; }</pre> Determine the total number of cycles required to execute the above loop on this pipeline processor, considering possible stage conflicts and pipeline stalls. The phase-time diagram is- <table><tr><th></th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th><th>7</th><th>8</th><th>9</th><th>10</th><th>11</th><th>12</th><th>13</th><th>14</th><th>15</th><th>16</th><th>17</th><th>18</th><th>19</th><th>20</th><th>21</th><th>22</th><th>23</th></tr><tr><td>S1</td><td>I1</td><td>I1</td><td>I2</td><td>I3</td><td>I3</td><td>I4</td><td>I1</td><td>I1</td><td>I2</td><td>I3</td><td>I3</td><td>I4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>S2</td><td></td><td></td><td>I1</td><td>I2</td><td>I2</td><td>I2</td><td>I3</td><td>I4</td><td>I4</td><td>I1</td><td>I2</td><td>I2</td><td>I2</td><td>I3</td><td>I4</td><td>I4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>S3</td><td></td><td></td><td></td><td>I1</td><td></td><td></td><td>I2</td><td>I2</td><td>I3</td><td>I4</td><td>I4</td><td>I1</td><td></td><td>I2</td><td>I2</td><td>I3</td><td>I4</td><td>I4</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>S4</td><td></td><td></td><td></td><td></td><td>I1</td><td></td><td></td><td></td><td>I2</td><td>I2</td><td>I3</td><td>I3</td><td>I3</td><td>I4</td><td>I4</td><td>I1</td><td>I2</td><td>I2</td><td>I3</td><td>I3</td><td>I3</td><td>I4</td><td>I4</td></tr></table> Phase-Time Diagram From here, number of clock cycles required to execute the loop = 23 clock cycles.	Instruction	S1	S2	S3	S4	I1	2	1	1	1	I2	1	3	2	2	I3	2	1	1	3	I4	1	2	2	2		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	S1	I1	I1	I2	I3	I3	I4	I1	I1	I2	I3	I3	I4												S2			I1	I2	I2	I2	I3	I4	I4	I1	I2	I2	I2	I3	I4	I4								S3				I1			I2	I2	I3	I4	I4	I1		I2	I2	I3	I4	I4						S4					I1				I2	I2	I3	I3	I3	I4	I4	I1	I2	I2	I3	I3	I3	I4	I4	[10]	CO5
Instruction	S1	S2	S3	S4																																																																																																																																																
I1	2	1	1	1																																																																																																																																																
I2	1	3	2	2																																																																																																																																																
I3	2	1	1	3																																																																																																																																																
I4	1	2	2	2																																																																																																																																																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23																																																																																																																													
S1	I1	I1	I2	I3	I3	I4	I1	I1	I2	I3	I3	I4																																																																																																																																								
S2			I1	I2	I2	I2	I3	I4	I4	I1	I2	I2	I2	I3	I4	I4																																																																																																																																				
S3				I1			I2	I2	I3	I4	I4	I1		I2	I2	I3	I4	I4																																																																																																																																		
S4					I1				I2	I2	I3	I3	I3	I4	I4	I1	I2	I2	I3	I3	I3	I4	I4																																																																																																																													